

Plan de Implementación: Sistema Modular y Escalable

Proyecto: AI-videoCreator Refactoring

Fecha: 2026-02-02

Objetivo: Transformar en sistema cloud-ready con prompts configurables y auto-generación de contenido

Resumen Ejecutivo

Refactorización completa del sistema AI-videoCreator para lograr:

1.  **Sistema de Templates de Prompts** - Prompts externos con variables, sin hardcoding
 2.  **Generación Automática de Topics** - LLM sugiere temas coherentes
 3.  **Videos Largos e Interactivos** - 180s con preguntas estilo Dora
 4.  **Reorganización Estructural** - Mejor arquitectura modular
 5.  **Configuración Cloud-Ready** - Totalmente externalizada
-

Consideraciones Importantes

[!IMPORTANT]

Cambio de Estructura: Scripts de prueba movidos a `src/testing/`.

Actualizar referencias existentes.

[!IMPORTANT]

Nuevo Sistema de Configuración: Prompts ahora en `prompts.json`. Permite cambios sin tocar código.

[!WARNING]

Duración Aumentada: Videos de 60s → 180s. Mayor costo de APIs y tiempo de generación.

[!TIP]

Interactividad Configurable: Flag `interactivity_enabled` controla preguntas/pausas por nicho.



Arquitectura del Sistema

Diagrama de Componentes Principales

```
graph TB
    subgraph "Pod Configuration"
        CONFIG[config.json<br/>Settings & Metadata]
        PROMPTS[prompts.json<br/>Template System]
        MEMORY[universe_memory.json<br/>Episode History]
    end
    end
```

```

subgraph "Core Engines"
    TOPIC[TopicEngine<br/>Auto-generate Ideas]
    SCRIPT[ScriptEngine<br/>Generate Scripts]
    VISUAL[VisualEngine<br/>Generate Images]
    AUDIO[AudioEngine<br/>Generate Voice]
    VIDEO[VideoEngine<br/>Assemble Final]
end

subgraph "Utilities"
    PROMPT_MGR[PromptManager<br/>Template Rendering]
    MEM_MGR[MemoryManager<br/>Context Management]
end

CONFIG --> TOPIC
CONFIG --> SCRIPT
PROMPTS --> PROMPT_MGR
PROMPT_MGR --> TOPIC
PROMPT_MGR --> SCRIPT
MEMORY --> MEM_MGR
MEM_MGR --> TOPIC
MEM_MGR --> SCRIPT

TOPIC --> SCRIPT
SCRIPT --> VISUAL
VISUAL --> VIDEO
SCRIPT --> AUDIO
AUDIO --> VIDEO

style TOPIC fill:#90EE90
style PROMPT_MGR fill:#90EE90
style CONFIG fill:#FFD700
style PROMPTS fill:#FFD700

```

Flujo de Ejecución: Modo Auto-Topic

```
sequenceDiagram
    participant User
    participant Main
    participant TopicEngine
    participant PromptManager
    participant ScriptEngine
    participant MemoryManager
    participant VisualEngine
    participant AudioEngine
    participant VideoEngine

    User->>Main: --auto-topic --pod kids_story
    Main->>TopicEngine: generate_topic()
    TopicEngine->>MemoryManager: get_all_episodes()
    MemoryManager-->>TopicEngine: [previous episodes]
    TopicEngine->>PromptManager: render_template('topic_generation')
    PromptManager-->>TopicEngine: rendered prompt
    TopicEngine->>TopicEngine: Call Gemini API
    TopicEngine-->>Main: topic={title, description}

    Main->>ScriptEngine: generate_script(topic)
    ScriptEngine->>PromptManager: render_template('script_generation')
    ScriptEngine->>MemoryManager: get_context_summary()
    ScriptEngine->>ScriptEngine: Call Gemini API
    ScriptEngine-->>Main: script={scenes, audio, visuals}

    Main->>VisualEngine: generate_visuals(script)
    VisualEngine->>PromptManager: get visual templates
    VisualEngine->>VisualEngine: Try Imagen-3.0
    VisualEngine->>VisualEngine: Fallback if needed
    VisualEngine-->>Main: visual_paths[]

    Main->>AudioEngine: generate_narration(script)
    AudioEngine->>AudioEngine: Call ElevenLabs TTS
    AudioEngine-->>Main: audio_paths[]

    Main->>VideoEngine: assemble_video(...)
    VideoEngine->>VideoEngine: MoviePy assembly
```

```
VideoEngine-->>Main: final_video.mp4

Main-->>MemoryManager: save_episode(script)

Main-->>User:  Video created!
```

Nueva Estructura de Proyecto

Antes vs Después

ANTES:

```
AI-videoCreator/
├─ check_image_gen.py      ✗ En raíz
├─ list_models.py          ✗ En raíz
├─ list_voices.py          ✗ En raíz
├─ pods/kids_story/
│   └─ config.json         ⚠ Básico
└─ src/engines/
    ├─ script_engine.py    ⚠ Prompts hardcoded
    └─ visual_engine.py    ⚠ Solo Pillow
```

DESPUÉS ✨:

```
AI-videoCreator/
├─ pods/kids_story/
│   ├─ config.json         ✓ Expandido
│   ├─ prompts.json       NEW Templates
│   └─ universe_memory.json
├─ src/
│   └─ engines/
│       ├─ topic_engine.py  NEW Auto-topics
│       ├─ script_engine.py ✓ Usa templates
│       └─ visual_engine.py ✓ Gemini Imagen
```

```
|   |   |─ utils/
|   |   |   |─ prompt_manager.py  NEW Template system
|   |   |   |─ memory_manager.py  ✓ Extended
|   |   |─ testing/               NEW Organized
|   |   |   |─ check_image_gen.py  ✓ Moved
|   |   |   |─ list_models.py      ✓ Moved
|   |   |   |─ list_voices.py      ✓ Moved
|   |─ main.py                   ✓ New CLI flags
```



Componentes Implementados

1. PromptManager (NUEVO)

Archivo: `src/utils/prompt_manager.py`

Funcionalidad:

- Carga templates desde `prompts.json`
- Reemplazo de variables con `{placeholder}` syntax
- Validación de variables requeridas
- Caché de templates para performance

Ejemplo de uso:

```
pm = PromptManager('pods/kids_story/prompts.json')
prompt = pm.render_template(
    'script_generation',
    topic='Tico aprende a compartir',
    duration_seconds=180,
    characters='Tico, Lila'
)
```

2. TopicEngine (NUEVO)

Archivo: `src/engines/topic_engine.py`

Funcionalidad:

- Genera topics automáticamente usando LLM
- Analiza episodios previos para evitar repetición
- Sugiere continuaciones coherentes
- Metadata educativa (valor, emoción objetivo)

Ejemplo de uso:

```
engine = TopicEngine('pods/kids_story/config.json')
topics = engine.generate_topics(count=5)
# [{title: "...", description: "...", educational_value: "..."}]
```

3. ScriptEngine (MEJORADO)

Archivo: `src/engines/script_engine.py`

Cambios:

- ☒ Usa `PromptManager` en lugar de prompts hardcodeados
- ☒ Videos de 180 segundos (configurable)
- ☒ Estructura narrativa: Intro → Desarrollo → Clímax → Conclusión
- ☒ Preguntas interactivas con pausas (configurable)

Ejemplo de escena interactiva:

```
{
  "scene_number": 8,
  "narrative_phase": "climax",
  "visual_prompt": "Tico mirando al espectador con expresión pensativa",
  "audio_text": "¿Tú qué harías en el lugar de Tico?",
  "character": "Narrator",
```

```
"is_interactive_question": true,  
"pause_for_answer": 3,  
"duration_est": 8  
}
```

Archivos de Configuración

prompts.json (Ejemplo)

```
{  
  "script_generation": {  
    "system_role": "Eres un narrador experto en cuentos infantiles educativos",  
    "user_template": "Tema: {topic}\nDuración: {duration_seconds}s\nPersonajes: {characters}\nEscenario: {setting}\n",  
    "output_format": {  
      "title": "string",  
      "moral": "string",  
      "scenes": []  
    }  
  },  
  "topic_generation": {  
    "system_role": "Eres un creativo de contenido infantil.",  
    "user_template": "Genera {num_topics} ideas para la serie '{series_name}'",  
    "output_format": {  
      "topics": [{  
        "title": "string",  
        "description": "string",  
        "educational_value": "string"  
      }]  
    }  
  }  
}
```


config.json (Expandido)

```
{
  "series_name": "Las Aventuras de Tico",
  "series_description": "Tico aprende valores en cada episodio",
  "target_audience": "niños de 3 a 7 años",

  "video_settings": {
    "duration_seconds": 180,
    "min_scenes": 12,
    "max_scenes": 20,
    "interactive_questions": 2,
    "interactivity_enabled": true
  },

  "visual_generation": {
    "gemini_models_priority": [
      "imagen-3.0-generate-001",
      "imagen-2.0-generate-001"
    ],
    "fallback_to_mock": true
  },

  "topic_generation": {
    "enabled": true,
    "auto_generate_count": 5
  }
}
```



Nuevas Funcionalidades CLI

Comandos Disponibles

```
# 1. Topic Manual
python -m src.main --topic "Tico y la honestidad" --pod kids_story

# 2. Auto-Topic (Automatizado)
python -m src.main --auto-topic --pod kids_story

# 3. Generar Ideas (sin video)
python -m src.main --generate-topics 10 --pod kids_story
```

Diagrama de Flujo CLI

```
graph TD
    START[User ejecuta comando] --> CHOICE{Qué modo?}
    CHOICE -->|--topic 'X'| MANUAL[Modo Manual]
    CHOICE -->|--auto-topic| AUTO[Modo Auto]
    CHOICE -->|--generate-topics N| IDEAS[Solo Ideas]
    MANUAL --> GEN_SCRIPT[Generar Guion]
    AUTO --> TOPIC_ENG[TopicEngine]
    TOPIC_ENG --> GEN_SCRIPT
    IDEAS --> TOPIC_ENG2[TopicEngine]
    TOPIC_ENG2 --> DISPLAY[Mostrar Topics]
    DISPLAY --> END[Fin]
    GEN_SCRIPT --> VISUAL[Generar Visuales]
    VISUAL --> AUDIO[Generar Audio]
    AUDIO --> VIDEO[Ensamblar Video]
    VIDEO --> SAVE[Guardar en Memoria]
    SAVE --> END
    style AUTO fill:#90EE90
```

```
style IDEAS fill:#87CEEB  
style END fill:#FFD700
```



Plan de Verificación

Test 1: Prompt Manager

```
python -c "from src.utils.prompt_manager import PromptManager; \  
pm = PromptManager('pods/kids_story/prompts.json'); \  
print(pm.render_template('script_generation', topic='Test', duration_secon
```

Esperado: Template renderizado con variables reemplazadas

Test 2: Topic Generation

```
python -m src.main --generate-topics 3 --pod kids_story
```

Esperado:

- 3 topics únicos
- Metadata completa
- No repetición vs memoria

Test 3: Video Completo Auto

```
python -m src.main --auto-topic --pod kids_story
```

Esperado:

- Topic auto-generado
- Video ~180s
- 2+ preguntas interactivas
- Imágenes de calidad



Métricas de Impacto

Aspecto	Antes	Después	Mejora
Duración	60s fijos	180s config	+200%
Prompts	Hardcoded	JSON editable	⚡ Instant
Topics	Manual	Auto (LLM)	🤖 Automated
Interactividad	No	Sí (opcional)	✨ New
Cloud-ready	Parcial	Total	☁ Ready
Escalabilidad	Baja	Alta	📈 Multi-pod



Casos de Uso Reales

Caso 1: Producción Diaria Automatizada

```
# Cron job: Cada día a las 9am  
python -m src.main --auto-topic --pod kids_story
```

Resultado: Video nuevo sin intervención manual

Caso 2: Cambiar Tono Dramático

Editar `prompts.json` :

```
{"system_role": "Eres un narrador DRAMÁTICO y TEATRAL..."}
```

Ejecutar:

```
python -m src.main --topic "Tico en la tormenta" --pod kids_story
```

Resultado: Guion con tono dramático, SIN cambiar código

Caso 3: Planificar Contenido Semanal

```
python -m src.main --generate-topics 7 --pod kids_story > ideas_semana.txt
```

Resultado: 7 topics para revisar y programar



Próximos Pasos Sugeridos

1. **✓ COMPLETADO** - Implementación base
 2. **Mejoras Visuales** - Integración SJinn AI para personajes
 3. **Multi-idioma** - Soporte EN/PT en prompts
 4. **Docker** - Containerización para cloud
 5. **YouTube API** - Upload automatizado
 6. **Dashboard** - UI para gestión de pods
-



Documentación Relacionada

- `walkthrough.md` - Guía completa de cambios realizados
 - `refactoring_comparison.md` - Comparación visual antes/después
 - `task.md` - Checklist de progreso
 - `README.md` - Guía de usuario actualizada
-



Sistema listo para producción escalable y cloud deployment!

Nota: Los diagramas Mermaid se muestran como código. Para visualizarlos, abra el archivo .md en GitHub o VS Code.